

CSE211s Introduction to Embedded Systems

Spring 2026

Instruction Set

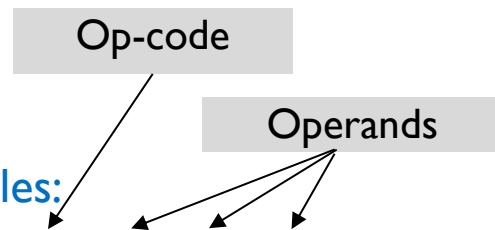
Topics

- Instruction Format
- Instruction Set Architecture __ ISA
- Example Instruction Sets: ARM, Thumb and Thumb-2
- Instruction Set Categories
 - *Data Transfer*
 - *Data Processing*
 - *Flow Control*
 - *Others...*

Instruction Format

- **All** instructions have **operation**, field (op-code).
- **Usually** instructions have reference to operands: one, two, three or even four operands:

operation <operand1> <,operand2> <,operand3> <,operand4>



- **Examples:**

- **ADD R0, R1, R2;** Add registers: $R0 = R1 + R2$
- **MOV R1, #20;** Move a decimal value 20 to register R1
- **CMP R1, R2;** Compare: Set condition flags based on result of $R1 - R2$ (R1 and R2 not changed)
- **LDR R1, [R3];** Load register R1 from memory at address in R3 (R3 used as a pointer. Note the square brackets)

Remember in a RISC CPU, an operand should be ready in the CPU. No operations with memory except for LOAD, STORE and stack operations. Only LOAD and STORE instructions have memory operands.

For all other instructions, an operand can be

- A register (or a shifted register)
- An immediate value within the instruction

Labels

- Sometimes we need to give the instructions a label to be able to reference it by other pieces of the code:

<label> operation <,operand1> <,operand2> <,operand3> <,operand4>

- Examples:

(1)

```
MOV R2, #1
again MUL R2, R2, R0
SUBS R1, R1, #1
BNE again
```

(2)

```
LOOP BEQ LOOP; Wait for non-zero condition.
How would such condition happen to exit LOOP?
```

Where Can the Operand be Located?

1. In the CPU

In a general-purpose register

- Destination: R_d
- Source: R_m, R_n

How the instruction is encoded

ADDS <Rd>,<Rn>,<Rm>

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	0	1	1	0	0	Rm			Rn			Rd		

The register number is encoded in 3 bits. Therefore, 16-bit Thumb instructions can reach only low eight registers; R0-R7

$R_m + R_n \longrightarrow R_d$

2. In memory

- Only for load, store, push and pop instructions as we have a RISC processor.

LDR <Rt>,<Rn>,<Rm>

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	1	0	1	1	0	0	Rm			Rn			Rt		

Pointer to memory

$[R_m + R_n] \longrightarrow R_d$

3. Within the instruction itself as an immediate value; *imm*

ADDS <Rdn>,<imm8>

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	1	1	0	Rdn			imm8							

$R_{dn} + \text{imm8} \longrightarrow R_{dn}$

Addressing Modes

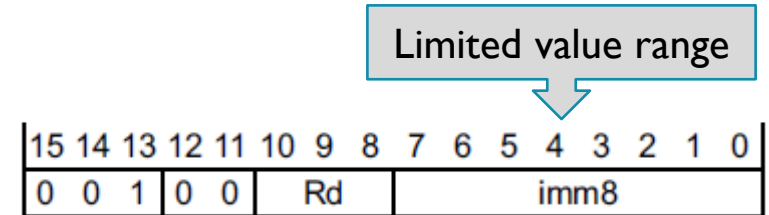
- The way the instruction references the operand(s) is called the **addressing mode**.

Cortex-M has three modes of addressing:

1. Immediate addressing:

The instruction includes the operand itself (Immediate value)

Example: ADD R1, #4 Adds 4 to R1.



2. Register addressing:

Operand is in a CPU register $\Rightarrow$ referenced by register's name (R1, R2, ...) and require no memory data fetches.

Example: ADD R1, R2 Performs R1+R2 and places the sum in R1.

3. Register Indirect (Base or Indexed) / Relative addressing:

The instruction uses one of the CPU 32-bit registers as a pointer to the operand in memory.

More information, like *offset*, can be included on how to calculate the operand's effective address.

Examples: LDR R2, [R1] Loads R2 from memory location whose address is in R1 (base register)
 LDR R3, [PC, #100] Loads R3 with a **constant operand** from ROM address PC+100
 B [PC, 4] Branch to the **instruction** whose address is PC+4 (4 locations ahead)

Note: Square brackets indicates register indirect addressing.

Memory Addressing

A Problem

The memory needs 32-bit address to access an operand or fetch an instruction.

Problem:

A 32-bit address cannot fit in a 16-bit Thumb or 32-bit Thumb-2 instruction.

Solution:

Use a 32-bit register to hold the address to point to the operand in memory.

- The address value should be prepared in a 32-bit register, used here as a **base register or index**, before it is used by the instruction that references the operand.

Optionally, an offset, which is a value in a CPU register or an immediate value within the instruction itself, can be added to the base register to form the effective address.

Base address with offset facilitate referencing elements in arrays and structures.

Memory Addressing Modes

Register Indirect Addressing

Forming memory address:

LDR Rd, [Rn<,offset>] *Use square brackets*

- Rn is called the base register (or index)
- The <offset> field is optional.
It can be either:
 1. An immediate value, or
 2. A value in another register to be added to Rn to form the effective address.A register shift also can be specified.

Memory Addressing Modes

Register Indirect Addressing Examples

Forms of base addressing mode:

1. Base/Offset addressing: $[R_n, <offset>]$

The Memory address is $[R_n] + offset$. R_n is unchanged.

Examples: LDR R0, [R1]; load R0 from location 0x1000
 LDR R0, [R1, #4]; load R0 from location 0x1004
 LDR R0, [R1, R2]; load R0 from location 0x1004

2. Pre-indexed addressing: $[R_n, offset]!$

- The Memory address is $[R_n] + offset$.
- $[R_n] + offset$ is written back into R_n .

Example: LDR R0, [R1, #4]!; load R0 from location 0x1004, then change [R1] to 0x1004.

3. Post-indexed addressing: $[R_n], offset$

- The Memory address is $[R_n]$.
- $[R_n] + offset$ is written back into R_n .

Example: LDR R0, [R1], #4; load R0 from location 0x1000, then change [R1] to 0x1004.

In each example assume register contents:

$R1 = 0x1000$

$R2 = 0x4$

In each case, R_n is the base register and *offset* can be:

- An immediate constant.
- An index register, R_m , or a shifted index register, such as $R_m, LSL \#shift$.

In a C for-loop we can use:

pre-increment ++i

post-increment i++

Instruction Set Summary

Instruction Category	Instruction Type	Instructions
Data Transfer	Move	MOV
	Load/Store	LDR, LDRB, LDRH, LDRSH, LDRSB, LDM, STR, STRB, STRH, STM
	Stack operations	PUSH, POP
Data Processing (Arithmetic & Logical Operations)	Add, Subtract, Multiply	ADD, ADDS, ADCS, ADR, SUB, SUBS, SBCS, RSBS, MULS
	Compare	CMP, CMN
	Logical	ANDS, EORS, ORRS, BICS, MVNS, TST
	Shift and Rotate	LSLS, LSRs, ASRS, RORS
	Extend	SXTH, SXTB, UXTH, UXTB
	Reverse	REV, REV16, REVSH
Flow Control	Conditional branch	IT, B, BL, B{<c>}, BX, BLX
Processor State	Processor State	SVC, CPSID, CPSIE, SETEND, BKPT
No Operation	No Operation	NOP
Others		

Move (Register or immediate), No reference to memory

- **MOV:**

Copy data from source register (or immediate data) to destination register. *No update to condition flags*

- **MOVS:**

Copy data from source register (or immediate data) to destination register. *Update condition flags*

Possible addressing modes: *register or immediate data.*

<i>Examples:</i>	MOV R0, R1 ;	copy content of R1 to R0	<i>Register Addressing</i>
	MOVS R0, R1 ;	copy content of R1 to R0, update N and Z flags	<i>Register Addressing</i>
	MOVS R0, #200 ;	Rd receives the decimal value 200, N=0, Z=0, C=0	<i>Immediate Addressing</i>

Load/Store Needs Memory Addressing

- Most load and store instructions use base register, possibly with an offset, to point to memory.

A register should be initialized with the proper address before being used as a base register.

Examples:

LDR R1, [R0]; Memory address = content of R0, base register R0 unchanged

LDR R1, [R0, #8]; Memory address = content of R0 + 8, R0 not changed

LDR R3, [R1, R2]; Memory address = content of R1 + content of R2

STR R3, [R1, R2, LSL #2]; Memory address = content R1 + (content R2 x 4)

Other forms of base addressing with offset are available with some assemblers.

.

Loading/Storing Smaller Data Sizes

Loading a byte or half-word in a 32-bit register requires padding or extension.

What do we put in the upper bits of the register?

- Load/Store byte or half word:

LDRB R1, [R0, offset] ; load one byte (zero extend)

LDRH R1, [R0, offset] ; load half word (zero extend)

STRB R1, [R0, offset] ; store one byte

STRH R1, [R0, offset] ; store half word (16 bits)

- Load byte or half word: (sign extend)

LDRSB R1, [R0, offset] ; load one byte (sign extend)

LDRSH R1, [R0, offset] ; load half word (16 bits) (sign extend)

Example: How do we extend the byte 0x80 into a full word while keeping the numerical value?

- Unsigned? Then 0x80 = +128, so zero-pad to extend to word 0x0000_0080 = 128
- Signed? Then 0x80 = -128, so sign-extend to word 0xFFFF_FF80 = -128

Stack Operations

PUSH & POP

■ PUSH:

Push some or all of registers (R0-R7, LR) to stack, **decrement** SP by 4×number of registers saved.

■ *Syntax:* PUSH {reglist}

The *reglist* specifies the set of registers to be saved.

The registers are stored in sequence, the lowest-numbered register to the lowest memory address, through to the highest-numbered register to the highest memory address.

■ *Example:* PUSH {R1, R2, LR};

Save R1, R2, and LR on stack, decrement SP by 12.

R1 is saved at address [SP]–12

R2 is saved at address [SP]–8

LR is saved at address [SP]–4

Pushing LR saves the return address in case a function calls another function.

■ POP:

Pop some or all of registers (R0-R7, PC) from stack, **increment** SP by 4×number of registers restored.

■ *Syntax:* POP {reglist}

Always pops registers in same order (opposite of pushing).

The lowest-numbered register is loaded from the lowest memory address, through to the highest-numbered register from the highest memory address.

■ *Example:* POP {R1, R2, PC};

Load R1, R2, and PC from stack, increment SP by 12.

R1 is loaded from address [SP]–12

R2 is loaded from address [SP]–8

PC is loaded from address [SP]–4

If the PC is specified in *reglist*, the instruction loads PC from stack and causes a branch to the address loaded into the PC (return address).

Arithmetic & Logical Instructions

ADD/SUB

- ADD/SUB: Add/Sub register, or immediate value, result in a destination register. (update condition flags with S suffix.)
- *Examples:*

ADD R2, R0, R1; Add R1 to value in R0, result in R2

SUBS R0, R1; Subtract R1 from R0, result in R0 , update flags

ADDS R1, #5; Add 5 to R1, update flags

AND/OR/EOR

- Bitwise AND/ORR/EOR register, or immediate value, result in destination register, (update flags with S suffix.)
- *Examples:*

AND R2, R0, R1; R1 AND R0, result in R2

ANDS R2, R0, R1; R1 AND R0, result in R2 , update flags

ORRS R0, R1; R1 OR R0, result in R0, update flags

There are add with carry ADC and subtract with carry SBC instructions used for multi-precision arithmetic.

- By default, data processing instructions do not affect the condition flags.
To cause the condition codes (flags) in APSR to be updated, a data processing instruction can be postfix with the S symbol, which sets the S bit in the instruction encoding.
- Condition flags are set by comparison (CMP) and test (TST) instructions.

Changing Program Flow - Branches

Normal program flow:

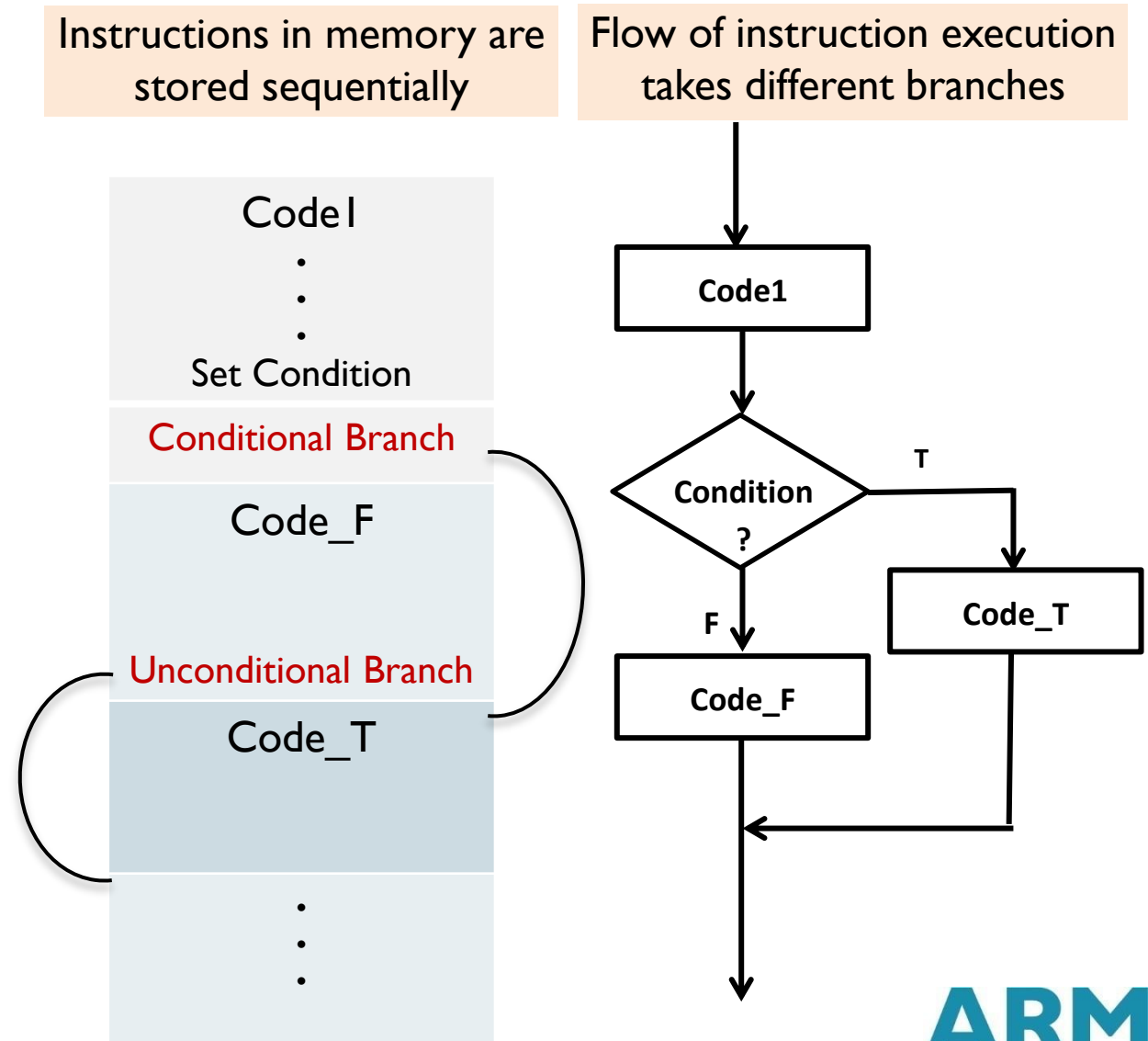
- Instructions are stored in memory sequentially.
- Instructions are executed sequentially.
- The PC is incremented by the size of the current instruction to point to the next instruction.

The need to change program flow:

- IF, IF-ELSE, and SELECT statements
- Loops like FOR or WHILE
- Calling a function and return

A **conditional branch** instruction is needed to branch to **Code_T** when **Condition** is true.

An **unconditional branch** instruction is needed at the end of **Code_F** to avoid overrunning **Code_T**.



Changing Program Flow - Branches

- A **conditional branch** instruction is needed to branch when a **Condition** is true.
- An **unconditional branch** instruction is needed to branch regardless of any condition.
- Branch and link (BL) and Branch and link with exchange (BLX).

Changing Program Flow - Branches

We need to specify a 32-bit addresses to branch to an instruction in memory.

Problem:

A 32-bit instruction address cannot fit in the operand field in a 16- or 32-bit instruction with the op-code.

Solution:

The 32-bit instruction address is formed in one of two ways:

1. Relative to the PC:

Use immediate value in the operand field of the instruction to be added to the PC.

This way is used if the distance to the destination can be represented in the instruction's operand field.

2. Use register addressing.

Changing Program Flow – Unconditional Branches

Unconditional Branches: cause a branch to a target address:

1. Relative to the PC

Syntax: B <label>; Branch to the instruction labeled <label>.
Addressing is PC-relative: *Supports limited address space.*
The assembler calculates the required offset from the PC value of the B instruction to this label.

Example: LOOP B LOOP; infinite loop using PC-relative addressing.

2. In a register.

Syntax: BX <Rm>; Branch to the instruction whose address is in Rm. *Supports full 4 GB address space.*
LSB of target address must be set to 1 to ensure continued execution in Thumb state.

Example: BX R0; unconditional branch to location whose address is in R0.

Changing Program Flow – Conditional Branches

Conditional Branches: cause a branch to a target address only if *cond* is satisfied.

Syntax: B<cond> <label>;

<cond> is the condition for branching.

Example: BNE SKIP;

if Z=0 then branch to SKIP,
else continue with next instruction.

- Append <cond> to branch instruction (B) to make a conditional branch.

Examples: BEQ

BNE

BPL

⋮

- Full ARM instructions (not Thumb or Thumb-2) support conditional execution of arbitrary instructions.

Mnemonic extension	Meaning	Condition flags
EQ	Equal	$Z = 1$
NE	Not equal	$Z = 0$
CS ^a	Carry set	$C = 1$
CC ^b	Carry clear	$C = 0$
MI	Minus, negative	$N = 1$
PL	Plus, positive or zero	$N = 0$
VS	Overflow	$V = 1$
VC	No overflow	$V = 0$
HI	Unsigned higher	$C = 1$ and $Z = 0$
LS	Unsigned lower or same	$C = 0$ or $Z = 1$
GE	Signed greater than or equal	$N = V$
LT	Signed less than	$N \neq V$
GT	Signed greater than	$Z = 0$ and $N = V$
LE	Signed less than or equal	$Z = 1$ or $N \neq V$
None (AL) ^d	Always (unconditional)	Any

Changing Program Flow – Subroutines (Functions)

- A subroutine is a block of code that performs a task.

A subroutine may take some arguments and may return a result.

- You use branch instructions to
 - call a subroutine
 - return from subroutines.

Changing Program Flow – Subroutine Call

Use the Branch with Link instructions.

We have two ways:

1. Use the branch with link instruction with offset value *<label>*:

`BL <label>`

which means call the subroutine at *<label>*. This instruction does the following:

1. Save return address in LR.
2. Add the value of *<label>* to PC. → Limited address range

2. branch with link and exchange.

`BLX <Rm>` -

which means call subroutine at address in register Rm. → Full 4 GB address range.

1. Save return address in LR
2. Copy Rm to PC. The LSB of target address copied to PC from Rm.

Changing Program Flow – Subroutine Return

Return from a subroutine

When a subroutine is called, the return address is placed in the link register (LR) by the (BL) or (BLX) instruction.

Therefore, control can be returned to the calling function by executing an unconditional branch via the LR register:

- BX LR - Return from subroutine

Problem:

If the called subroutine calls another subroutine, then the return address in LR will be overwritten by the second call and the link to the originally calling code is lost!

To prevent such situation, ***a subroutine that calls another subroutine***, the LR register should be saved on the stack by a PUCH instruction before calling the second subroutine.

In this case, the return instruction will be a POP to the PC, not a BX LR instruction, which pops the return address saved before on the stack into the program counter.

Note that if other registers were pushed onto the stack, the POP instruction may have a list of multiple registers.

Subroutine Call & Return Example

```
start  MOV    r0, #10                ; Set up parameters to pass to the subroutine
      MOV    r1, #3
      BL     doadd                  ; Call subroutine
      .
      .
      .
stop   MOV    r0, #0x18              ; Return to supervisor
      LDR    r1, =0x20026
      SVC    #0x123456

doadd  ADD    r0, r0, r1             ; Subroutine code
      BX     lr                    ; Return from subroutine
      END                                ; Mark end of file
```

Extra Readings

Instruction Set Architecture: ISA

In general, an ISA of a CPU defines the supported:

- Instruction set (operations recognized by the CPU)
- Data types
- Registers
- The hardware support for managing main memory and input/output

ARM, Thumb and Thumb-2 Instruction Sets

- ARM instructions are optimized for resource-rich high-performance computing systems such as mobile applications.
 - Deeply pipelined processor, high clock rate, wide 32-bit memory bus

But low-end embedded computing systems are different:

- They have slower clock rates, shallow pipelines
 - Different cost factors – e.g. code size matters much more, bit and byte operations are critical
- Modifications to ARM Instruction Set Architecture (ISA) to fit low-end embedded computing

1995

Thumb instruction set

Is a compact **16-bit** encoding for a *subset* of the ARM instruction set.

- Reduces memory requirements but reduces performance slightly

2003

Thumb-2 instruction set

Is the Thumb set with a few 32-bit instructions added to give the instruction set more breadth.

- Improves speed with little memory overhead

- CPU decodes instructions based on whether in Thumb or ARM state - controlled by T bit in EPSR.

Cortex-M0+ Instruction Set

- Cortex-M0+ core implements ARMv6-M Thumb 56 instructions, , including a few 32-bit instructions that use Thumb-2 technology such as BL, DMB, DSB, ISB, MRS and MSR
- The core is always in Thumb state controlled by T bit in EPSR and indicated by program counter being odd ($PC_0 = 1$)
 - Branching to an even address will cause an **exception**.
 - Most 16-bit instructions can only access low registers (R0-R7), but a small number of 16-bit instructions can access high registers (R8-R15)
 - Conditional execution of instructions is supported only for branch instructions.
 - Half-word aligned instructions ($PC_0 = 1$)
 - 32-bit address space

In-Register Size Extension

- Can also extend byte or half-word already in a register
 - Signed or unsigned (zero-pad)
- How do we extend 0x80 into a full word?
 - Unsigned? Then $0x80 = +128$, so zero-pad to extend to word $0x0000_0080 = 128$
 - Signed? Then $0x80 = -128$, so sign-extend to word $0xFFFF_FF80 = -128$

	Signed	Unsigned
Byte	SXTB <Rd>,<Rm>	UXTB <Rd>,<Rm>
Half-word	SXTH <Rd>,<Rm>	UXTH <Rd>,<Rm>

Load/Store Multiple

- LDM (Load Multiple): Load registers from consecutive locations starting from base register:
 - *Syntax:* LDM Rn!, {reglist} ; ! Means update base register after. (by what value?)
LDM Rn, {reglist}
 - *Example:* LDM R0 , {R0,R3,R4} ; ! is not used because the base register R0 is in *reglist*
- STM (Store Multiple): Store registers to consecutive locations starting at [base register]
 - *Syntax:* STM Rn!, {reglist}
STM Rn, {reglist}
 - *Example:* STM R1! , {R2-R4,R6}; R1 will be updated by 16

These instructions do not change the flags.

Add Instructions (Add with Carry)

- ADC: Add register, or immediate value, the carry flag, and another register, result in the destination register.
- *Syntax:* `ADC{S} {Rd,} Rn, Operand2 ;` Add *Operand2* and value in *Rn*, together with the carry flag, ; result in *Rd* if specified, otherwise in *Rn*.

Used for multi-precision arithmetic:

Two instructions add two 64-bit integers:

- *Example:*

Add the 64-bit integer in R3-R2 64-bit integer to
64-bit integer in R1-R0, and
place the result in R5-R4:

`ADDSD r4, r0, r2 ;` adding the least significant words

`ADC r5, r1, r3 ;` adding the most significant words with carry from previous addition.

Subtract Instructions (Subtract with Carry)

- SBC: Subtract register, or immediate value, the carry flag, and another register, result in the destination register.
- *Syntax*: SBC{S} {Rd,} Rn, *Operand2* ; Subtract *Operand2* from Rn, if the carry flag is clear, the result is reduced by 1, result in Rd if specified, otherwise in Rn.

Used for multi-precision arithmetic:

Two instructions subtract two integers more than 32-bits.

- *Example*:

Subtract 96-bit integer in R11-R9 from

96-bit integer in R8-R6, and

place the result in R3-R5:

SUBS r3, r6, r9 ; subtract the least significant words

SBCS r4, r7, r10

SBC r5, r8, r11 ; subtract the most significant words

Multiply

- Multiply source registers, signed or unsigned, save least significant word of result in destination register, {update condition flags}.
- *Syntax:* MUL{S} {Rd,} Rn, Rm; $Rd = Rm \times Rn$, if Rd is omitted, Rn receives the result.
Rn must be different from Rd

Note: upper word of result is truncated!

Logical Operations

- Bit clear register: Bitwise AND register and *complement* of second register, result in destination register, update condition flags.
 - *Syntax:* BIC {Rd,} Rn, *operand2*; Rn AND NOT(*operand2*), result in Rd, {update condition flags if S added}
 - *Example:* BICS R2, R3; if value in R3 is 7, clear the least three bits in R2, and update condition flags.
- Test selected bits in a register, update condition flags by ANDing two registers, discarding result.
 - *Syntax:* TST Rn, *operand2*; Bitwise AND between Rn AND *operand2*, update condition flags, discard result.
 - *Example:* TST R2, R3; If value in R3 is 7, then Z flag is set if none of the least three bits in R2 is 1.
Affects Z, N, and C flags.

Compare

- Compare a register value to another register, or immediate value,
(*as if* subtracting second value from first), update condition flags, discard result.
 - *Syntax*: `CMP Rn, operand2`; Compare value in Rn to *operand2* (as if *operand2* is subtracted from Rn), update flags, discard result. Permitted values of *const* is 0-255 for encoding T1
 - *Example*: `CMP R5, #5`; Compare R5 to 5: $(R5 - 5)$, update flags according to result, discard result:
- Compare negative - **adds** values in Rn and *operand2*, update condition flags, discards result.
 - *Syntax*: `CMN Rn, Rm` ; Compare Rn to $-Rm$ ($Rn + Rm$), update flags according to result, discard result.
 - *Example*: `CMN R5, R6`; Add values in R5 and R6, update flags according to result, discard result.

Loading The Address into a Register (ADR Instruction)

- The ADR instruction forms a PC-relative address. It loads an address within a certain range from the instruction itself (ALIGN PC,4) into a register.

It adds an immediate value to the PC value, and writes the result to the destination register (base register)

- *Syntax:*

- 1. ADR Rd, *label* ; *Normal syntax*

label is the label of an instruction or literal data item whose address is to be loaded into Rd.

The assembler calculates the value of *label* in the instruction as the offset from this instruction.

- 2. ADD Rd, PC, #<const> *Alternative syntax*

Notice that the alternative syntax it is an ADD instruction not ADR.

Used to generate a target for a BX or BLX instructions, and to prepare address for a data item.

- *label* must be within the same code section as the ADR instruction. The assembler faults references to labels that are out of range.

The available range of addresses for the ADR instruction depends on the instruction set and encoding.

- ↳ For 16-bit encoding it is 0 to 1020 bytes and word-aligned. (You can use the ALIGN directive to ensure this)

Load Literal Value into Register

Assembly *pseudo-instruction*: Not a machine instruction

LDR Rd, =value

Assembler generates code to load Rd with value, and selects best approach depending on value:

- Load immediate: MOV instruction provides 8-bit unsigned immediate operand (0-255)
- Load and shift immediate values: Can use MOV, shift, rotate, sign-extend instructions
- Load from literal pool: (literal pool is a portion of memory embedded in the code to hold constant values).
 1. Place value as a 32-bit literal in the program's literal pool
 2. Use instruction LDR Rd, [PC, #offset] where offset indicates position of literal relative to PC

Shift and Rotate: Logical Shift

- Logical shift **left** a register value a number of bits, shift in zeroes on right, result in destination register, update flags.

- *Syntax:* LSLS Rd, Rm, #imm5

LSLS Rdn,Rm

- *Examples:* LSLS R0, R5, #3; Multiply value in R5 by 8, result in R0, update condition flags.

LSLS R5, R2; If R2 contains the value 3, this instruction multiplies R5 by 8.

- Logical shift **right** a register value a number of bits, shift in zeroes on left, result in destination register, update flags.

- *Syntax:* LSRS Rd, Rm, #imm5

LSRS Rdn, Rm

- *Examples:* LSLS R0, R5, #3; Divide value in R5 by 8, result in R0, update condition flags.

LSLS R5, R2 ; If R2 contains the value 3, this instruction divides R5 by 8.

Shift and Rotate: Arithmetic Shift

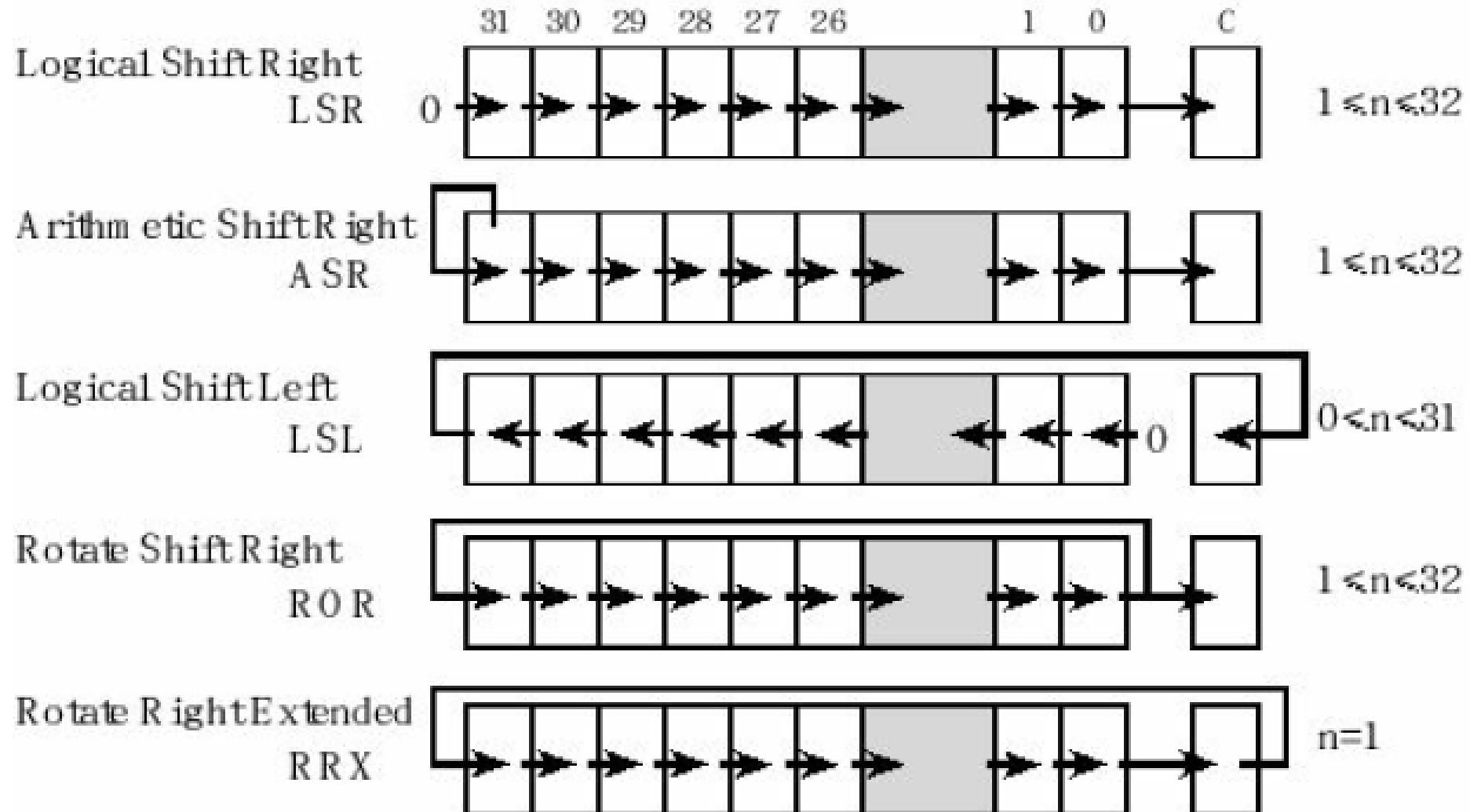
- Arithmetic shift **right** a register value a number of bits, shift in copies of sign bit (to maintain arithmetic sign), result in destination register, update flags.

- *Syntax:* ASRS Rd,Rm,#imm5

ASRS Rd, Rn, Rm

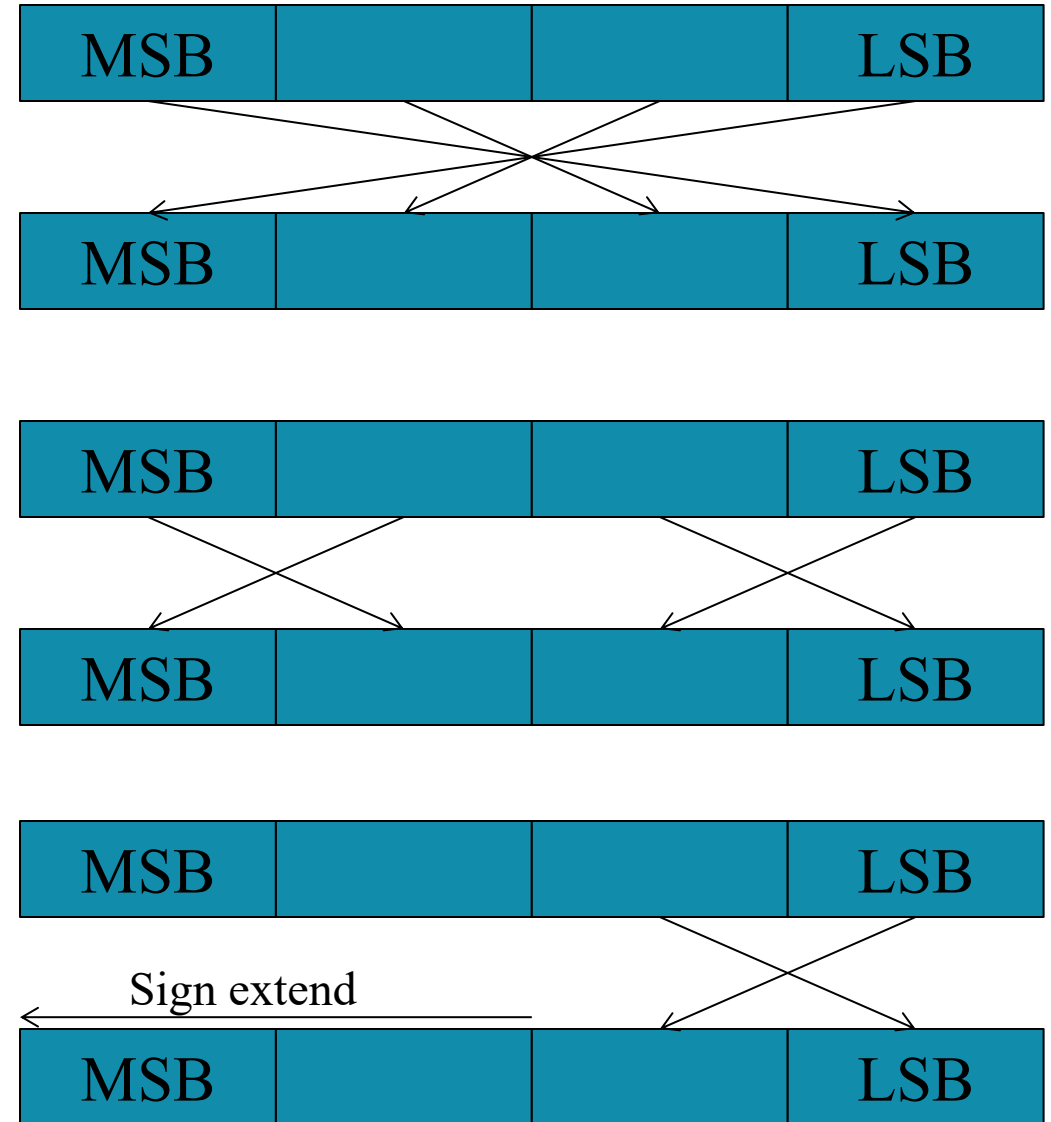
- *Example:* ASRS R0, R5, #4; Multiply value in R5 by 16, result in R0, update condition flags.
- Rotate right a register value a number of bits, The bits that are rotated off the right end are inserted into the vacated bit positions on the left, result in destination register, update flags.
- *Syntax:* RORS Rd, Rdn, Rm
- *Example:* RORS R0, R5, #4; if R5 contains the value 0x0003, R0 will have the value 0x3000.

Shift and Rotate



Reversing Bytes

- These instructions have no effect on condition flags.
- REV - reverse the byte order in a register.
 - Syntax:** REV Rd, Rm
 - Example:** REV Rd, Rm; if Rm has 0x1234, then Rd gets 0x4321.
- REV16 - reverses the byte order in each 16-bit halfword in a register.
 - Syntax:** REV16 Rd, Rm
 - Example:** REV16 Rd, Rm; if Rm has 0x1234, then Rd gets 0x2143.
- REVSH - reverses the byte order in the lower 16-bit halfword in a register, and sign extends the result to 32 bits.
 - Syntax:** REVSH Rd, Rm
 - Example:** REVSH R0, R1; if R1 has 0x8012, then R0 gets 0xFF21.



Special Register Instructions

- Move to Register from Special Register
 - MSR <Rd>, <spec_reg>
- Move to Special Register from Register
 - MRS <spec_reg>, <Rd>
- Change Processor State - Modify PRIMASK register
 - CPSIE - Interrupt enable
 - CPSID - Interrupt disable

Special register	Contents
APSR	The flags from previous instructions.
IAPSR	A composite of IPSR and APSR.
EAPSR	A composite of EPSR and APSR.
XPSR	A composite of all three PSR registers.
IPSR	The Interrupt status register.
EPSR	The execution status register. ^b
IEPSR	A composite of IPSR and EPSR.
MSP	The Main Stack pointer.
PSP	The Process Stack pointer.
PRIMASK	Register to mask out configurable exceptions. ^c
CONTROL	The CONTROL register, see <i>The special-purpose CONTROL register</i> on page B1-215.

Other

- **No Operation** - does nothing! - Can be used for code alignment purposes and for timing loops.
 - NOP
- **Breakpoint** - causes hard fault or debug halt - used to implement software breakpoints
 - BKPT #<imm8>
- **Wait for interrupt** - Pause program, enter low-power state until a WFI wake-up event occurs (e.g. an interrupt)
 - WFI
- **Wait for event** - Permits the processor to enter a low-power state until one of a number of events occurs.
 - WFE
- **Supervisor call** - Generates a call to a system supervisor. Same as software interrupt.
 - SVC #<imm>